AugCRNN-T: Augmented Convolutional-Recurrent Networks with Lexicon-Aware Post-Correction for Handwritten Word Recognition

***Abstract*—Recognising isolated handwritten words is difficult mainly because handwriting varies strongly from one writer to another. We study this problem on the IAM database under the Aachen writer-disjoint protocol, where the writers seen during training never appear in the test set. We propose AugCRNN-T, a convolutional-recurrent network trained from scratch with an augmentation strategy that imitates the physical act of writing — elastic stroke deformation and morphological changes of pen thickness — and combined with a lexicon-aware trigram language model applied after decoding. On the Aachen test set of 5,338 words, AugCRNN-T reaches 84.54% word accuracy, improving our un-augmented baseline by more than six percentage points; the gain is statistically significant and the confidence intervals of the two systems do not overlap. AugCRNN-T also outperforms two recent word-level convolutional-recurrent systems evaluated on easier, writer-dependent partitions of the same database, while matching the accuracy of an attention-based sequence-to-sequence model with a considerably simpler architecture. We additionally report which further refinements did *not* help — dictionary-constrained beam search, test-time augmentation and multi-model ensembling — and we quantify how much accuracy can be lost purely through software and hardware environment drift when a result is reproduced.**

***Index Terms*—handwritten text recognition, IAM database, convolutional-recurrent neural network, connectionist temporal classification, data augmentation, language model, writer-independent evaluation**

I. Introduction

Offline *handwritten text recognition* (HTR) is the task of converting a scanned image of handwriting into machine-readable text without access to the pen trajectory. Depending on how the input is segmented, an HTR system may operate on full pages, on text lines, or — as in this work — on isolated *word* images. Word-level recognition is the harder setting per sample, because the recogniser cannot exploit the surrounding words of a sentence to disambiguate an unclear glyph.

The IAM Handwriting Database [1] is the standard benchmark for English HTR, containing roughly 115 000 word images from 657 writers. We adopt the partition released by RWTH Aachen University [2] — referred to as the *Aachen split* throughout this paper — which guarantees that no writer contributes to more than one of the training, validation and test sets. This *writer-disjoint* protocol is stricter than a random split and is the realistic condition for deploying a recogniser on handwriting it has never seen.

Modern recognisers are trained with the *connectionist temporal classification* (CTC) objective [3], which removes the need for character-level alignment between the image and its transcription. Two model families dominate: convolutional-recurrent networks (CRNNs) trained with CTC [4]–[6], and attention-based sequence-to-sequence or transformer encoders [7], [8]. Attention models are powerful but typically demand more data, synthetic pre-training, or substantially larger capacity. Our position is that a compact CRNN remains highly competitive at the word level, provided that (i) the augmentation reflects how handwriting actually varies, and (ii) a lexical prior is applied after decoding.

We name our proposed system AugCRNN-T (Augmented CRNN with Trigram post-correction) and use this name consistently throughout the paper; Table I defines it together with the baseline variants it is compared against. Our contributions are:

- **A writing-aware augmentation strategy.** Elastic deformation and morphological pen-thickness perturbation, added on top of a conventional geometric/photometric pipeline, improve word accuracy by 6.48 pp over an otherwise identical baseline.
- **A lexicon-extended trigram post-corrector.** The recogniser's lexicon is enlarged from the 5.9 K training vocabulary to 238 K English word types, while the n-gram statistics remain estimated only on IAM training text.
- **A statistically grounded evaluation.** All results are reported on the writer-disjoint test set with Wilson 95% confidence intervals and McNemar exact paired tests, and are compared against three external word-level systems.
- **Reproducible negative results.** We show that word beam search, test-time augmentation and multi-model ensembling do not improve our system, and we measure the accuracy loss caused purely by environment drift during reproduction.

Section II groups the related literature into three families and positions our work within them. Section III describes the pipeline of Fig. 1. Section IV formalises the evaluation metrics and statistical tests. Section V reports the results, Section VI analyses them, and Section VII concludes.

II. Related Work

We organise prior work into three families according to how each system relates to ours: (i) *recurrent CTC recognisers*, which share our optical backbone but are almost always evaluated on text lines; (ii) *attention and transformer recognisers*, which target the same word-level task as ours but with a different, heavier decoding paradigm; and (iii) *lexicon and language-model decoders*, which address the same problem we address with our trigram post-corrector but from inside the decoding loop. For each family we state explicitly what we adopt and where we differ.

a) Recurrent CTC recognisers.: Graves and Schmidhuber [9] introduced multi-dimensional long short-term memory (LSTM) networks with CTC as the first end-to-end offline recogniser. Puigcerver [5] showed that a one-dimensional bidirectional LSTM (BiLSTM) stacked on a convolutional neural network (CNN) feature extractor matches multi-dimensional recurrence at a fraction of the cost, defining the canonical CRNN baseline that our architecture follows. Bluche and Messina [6] added gated convolutions, and Flor et al. [10] combined a CNN with a bidirectional gated recurrent unit (BGRU) and a language-model pipeline. We adopt this backbone family unchanged; our contribution is orthogonal, residing in the augmentation and in the post-decoding lexical prior.

b) Attention and transformer recognisers.: Sueiras et al. [11] and Kang et al. [7] introduced sequence-to-sequence recognisers with content-based attention for isolated IAM words. Kang et al. [12] later integrated a language model into the decoder itself through candidate fusion, and Kass and Vats [13] combined a ResNet encoder, attention decoding and transfer learning from scene-text corpora, reporting the strongest published word-level accuracy on the Aachen split. These four systems share our task, metric and split, and are compared against directly in Section V. Transformer architectures such as TrOCR [14] have since lowered line-level error rates further, but require pre-training on hundreds of millions of synthetic lines. Multimodal large language models have also been benchmarked on IAM [15]; those systems operate on full pages and rely on proprietary pre-training corpora, so they are not comparable to a from-scratch word recogniser.

c) Word-level CRNN systems and lexical decoders.: Two recent systems are directly comparable to ours in task, metric and model family. Dutta et al. [16] report an improved CNN–recurrent neural network (RNN) hybrid on isolated IAM words with a 22.86% word error rate (≈77.14% accuracy). Rajesh et al. [17] propose HWRCNet, a CNN–BiLSTM operating in the JPEG discrete cosine transform (DCT) domain, reaching 80.08% strict word accuracy on uncompressed images; we note that their headline figure of 89.05% is *word accuracy with flexibility* of two edit operations rather than strict accuracy. On the decoder side, word beam search (WBS) [18] injects a dictionary constraint directly into CTC beam search — the same lexical knowledge we exploit, but during rather than after decoding; we compare both strategies in Section V.

d) Positioning.: Both word-level baselines above are evaluated on random partitions of IAM that do not enforce writer disjointness (HWRCNet uses a 95:5 ratio). Because we evaluate on the stricter Aachen protocol, the margins we report in Section V are conservative estimates of the true methodological gap.

TABLE I
MODEL NAMING USED THROUGHOUT THE PAPER.

Name	BiLSTM	Params	Elastic/morph. aug.	Trigram
CRNN-S	2 layers	8.75 M	no	yes
CRNN-M	3 layers	15.46 M	no	yes
CRNN-L	4 layers	28.73 M	no	yes
AugCRNN	4 layers	28.73 M	yes	no
AugCRNN-T (proposed)	4 layers	28.73 M	yes	yes

III. Proposed System

Fig. 1 shows the complete processing chain, from the raw IAM word crops to the reported metrics, and marks the two stages that constitute our contribution.

A. Data and Protocol

We use the word-level IAM partition of the Aachen split [2]: 747 training forms, 116 validation forms and 336 test forms, with disjoint writer sets. Following common practice, only word crops whose segmentation is flagged `ok` are retained, giving 31 320 training, 1 646 validation and 5,338 test words. The label alphabet contains 78 symbols (upper- and lower-case Latin letters, digits and punctuation), plus the CTC blank.

B. Model Naming

To avoid ambiguity, Table I lists every configuration evaluated in this paper. AugCRNN-T denotes the proposed system and is the only model referred to as "ours" in the results; CRNN-S, CRNN-M and CRNN-L are our earlier baselines, differing only in recurrent depth and, for CRNN-L, in the absence of the proposed augmentation.

C. Optical Model

The encoder of AugCRNN-T is a seven-block convolutional feature extractor that collapses the input to a single row of 512-dimensional frames, followed by a four-layer bidirectional LSTM with 512 hidden units and a linear projection onto the 79 output classes; the network has 28.73 M parameters. Inputs are grayscale crops resized to 32×128, inverted so that ink is high-valued, and scaled to $[-1, 1]$. The network is trained with the CTC objective [3] using AdamW with learning rate $\eta = 7 \times 10^{-4}$ and weight decay 10^{-5}, five warm-up epochs followed by cosine decay, mixed precision, and batches of 128 on a single Tesla T4 graphics processing unit (GPU). Training stops when validation word accuracy has not improved for 15 epochs.

D. Writing-Aware Augmentation

Our baselines use a conventional pipeline: rotation ($\pm 7°$), translation ($\pm 5\%$), scaling (0.9–1.1), shear ($\pm 5°$), brightness and contrast jitter (0.85–1.15), Gaussian noise, gamma correction and random erasing.

AugCRNN-T extends this with two transforms chosen to reproduce genuine sources of handwriting variability rather than generic image perturbations:

- **Elastic deformation** (probability 0.5). A random displacement field is smoothed with a Gaussian kernel

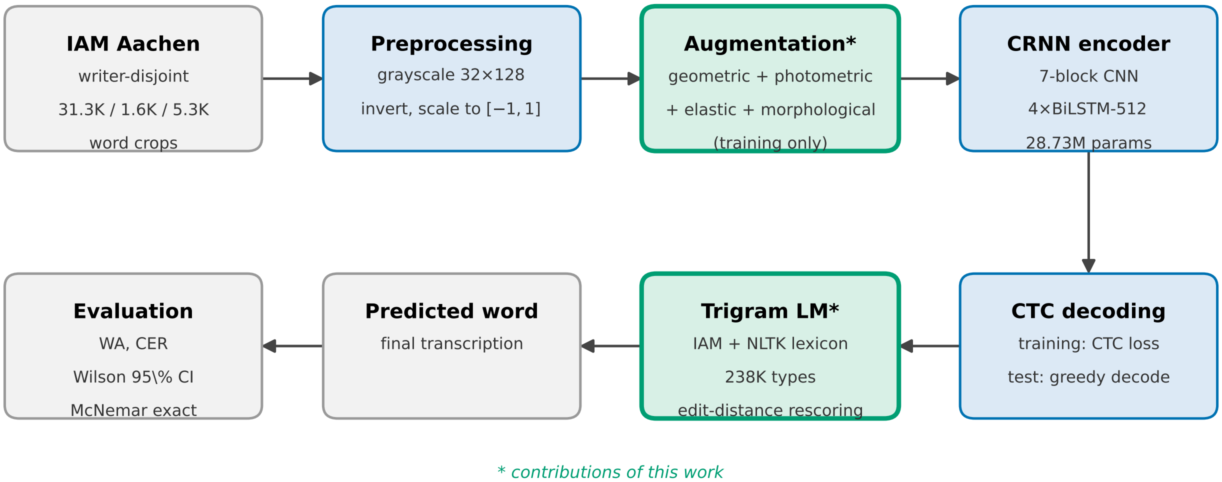

Fig. 1. Overview of AugCRNN-T. Word crops are normalised, augmented (training only), encoded by a CRNN and decoded with CTC; the decoded string is then corrected against a lexicon-extended trigram language model (LM). Green boxes mark the two contributions of this work. The evaluation stage reports word accuracy (WA), character error rate (CER) and Wilson confidence intervals (CI), all defined formally in Section IV.

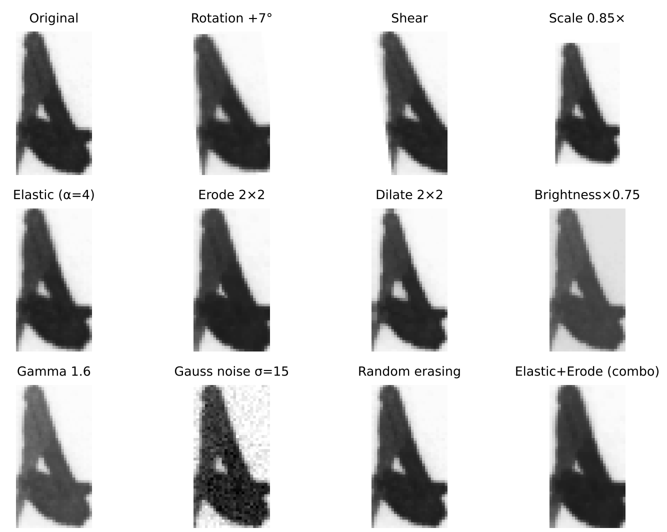

Fig. 2. Augmentation transforms on a training word. The last panel shows the compound elastic+morphological effect.

whose width is 8% of the larger image side and scaled by a factor drawn uniformly from $[2, 5]$; the crop is warped accordingly. This mimics pen jitter and the natural curvature drift of strokes.

- **Morphological perturbation** (probability 0.3). The crop is eroded or dilated with a $k \times k$ kernel, $k \in \{1, 2\}$, reproducing the difference between a fine and a broad nib.

The photometric ranges are widened accordingly (brightness and contrast to 0.70–1.35, gamma to 0.70–1.30, noise standard deviation to 0.05). Fig. 2 illustrates each transform on a representative training word.

E. Lexicon-Extended Trigram Post-Correction

After greedy CTC decoding, the hypothesis is checked against a closed lexicon. The lexicon is built from the 5 939 word types of the Aachen training transcriptions and extended with the Natural Language Toolkit (NLTK) English word list, yielding 238 506 types; crucially, the trigram counts themselves are estimated only from IAM training text, so no external corpus statistics leak into the model. A hypothesis inside the lexicon is accepted unchanged; otherwise it is replaced by the highest-scoring lexicon entry within a small edit-distance neighbourhood. This design keeps the corrector able to recover near-miss recognitions without biasing frequent-word statistics towards a foreign domain.

F. Alternative Decoders Evaluated

For completeness we also evaluate three refinements: WBS in NGrams mode (beam width 50, language model weight 0.7, 245 K-word corpus); test-time augmentation (TTA) with five affine views (identity, $\pm 2^\circ$ rotation, $0.96\times$ and $1.04\times$ scale); and softmax averaging over CRNN-S, CRNN-M and AugCRNN, with equal or 1:2:3 weights. Their results are reported in Section V.

IV. Experimental Setup

A. Evaluation Metrics

Let $\mathcal{T} = \{(\hat{y}_i, y_i)\}_{i=1}^{N}$ be the set of $N = 5{,}338$ predicted/reference word pairs. *Word accuracy* (WA) is the exact-match rate

$$\mathrm{WA} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}[\hat{y}_i = y_i], \tag{1}$$

and the word error rate (WER) follows as $\mathrm{WER} = 1 - \mathrm{WA}$. The *character error rate* (CER) aggregates Levenshtein distances $d(\cdot, \cdot)$ over the corpus,

$$\mathrm{CER} \;=\; \frac{\sum_{i=1}^{N} d(\hat{y}_i, y_i)}{\sum_{i=1}^{N} |y_i|}, \tag{2}$$

so that a single badly recognised long word cannot dominate the score.

B. Confidence Intervals

Because WA is a proportion estimated on a finite test set, we report Wilson 95% confidence intervals (CIs) rather than normal-approximation intervals, which are unreliable near the boundaries. For k correct predictions out of N and $z = 1.96$,

$$\mathrm{CI}_{95} = \frac{\hat{p} + \frac{z^2}{2N} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}, \qquad \hat{p} = \frac{k}{N}. \tag{3}$$

C. Significance Testing

Two systems evaluated on the same test set produce paired outcomes, so we use the exact McNemar test. Let b be the number of samples that the first system recognises correctly and the second does not, and c the converse. Under the null hypothesis the discordant pairs are distributed $\mathrm{Bin}(b + c, \frac{1}{2})$, giving the two-sided p-value

$$p \;=\; \min\left(1,\; 2 \sum_{i=0}^{\min(b,c)} \binom{b+c}{i} 2^{-(b+c)}\right). \tag{4}$$

We treat $p < 0.01$ as significant.

D. Reproducibility

Training was performed on Kaggle Notebooks with one NVIDIA Tesla T4 (15 GB), PyTorch 2.3.0, CUDA 12.1 and Python 3.10, and took approximately 119 minutes for 51 epochs. Code, split definitions, per-sample predictions and evaluation scripts are available at https://github.com/Ridvan013/CRNN-Handwriting-Recognition (branch `feature/aachen-v3-extended-trigram`); the trained weights exceed the repository file-size limit and are distributed separately.

V. Results

A. Effect of the Proposed Augmentation

Table II reports the headline comparison. AugCRNN-T reaches 84.54% word accuracy against 78.06% for CRNN-L, an identical network trained without elastic and morphological augmentation. The improvement of 6.48 pp is significant at $p < 10^{-30}$ by (4), and the Wilson intervals of the two systems are disjoint, so the gain cannot be attributed to test-set sampling noise. The character error rate falls by nearly two points in parallel, indicating that the augmentation improves the optical model itself rather than merely making the lexical corrector more effective.

Fig. 3 shows the corresponding training dynamics. The validation accuracy rises steeply for the first 30 epochs and then flattens, with the best checkpoint obtained at epoch 51; the un-augmented baseline converges earlier but to a lower plateau, consistent with the augmentation acting as a regulariser.

TABLE II
Effect of the Proposed Augmentation (Aachen Test Set, $N = 5{,}338$).

Model	WA (%)	CER (%)	Wilson 95% CI	ΔWA
CRNN-L (baseline)	78.06	11.16	[76.93, 79.15]	—
AugCRNN-T (proposed)	**84.54**	**9.21**	[83.55, 85.49]	+6.48 pp

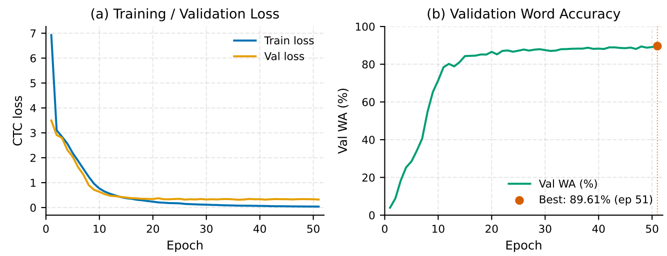

Fig. 3. Training dynamics of AugCRNN-T: (a) CTC losses, (b) validation word accuracy, best checkpoint marked in red.

B. Comparison with Prior Word-Level Systems

Table III compares AugCRNN-T against six external word-level systems and against our own baseline family. The upper block lists systems evaluated under the *same* Aachen writer-disjoint protocol, and is therefore the fairest comparison available. AugCRNN-T clearly outperforms the sequence-to-sequence recogniser of Sueiras et al. [11] (+8.3 pp) and the attention-based model of Kang et al. [7] (+2.0 pp). It is statistically indistinguishable from the two strongest published word-level systems: the language-model-augmented architecture of Kang et al. [12] (84.09%) and AttentionHTR [13] (84.60%, only 0.06 pp above ours). This parity is notable given the difference in machinery: AttentionHTR couples a ResNet encoder with a content-based attention decoder and transfers knowledge from 14.4 M scene-text and Imgur5K images, whereas AugCRNN-T is a plain CRNN with CTC trained from scratch on 31 K IAM word crops alone. Matching that word accuracy without external pre-training or an attention decoder is the central empirical claim of this paper.

The picture differs at character level, where the attention-based systems retain a clear advantage (5.79–6.50% CER against our 9.21%). We attribute this to their autoregressive decoders, which model inter-character dependencies explicitly, whereas our conditionally independent CTC outputs are corrected only afterwards and only at the whole-word level: the lexical prior can repair a word or leave it unchanged, but it cannot reduce the character error of a word that remains outside the lexicon.

The lower block reports two systems measured on random, writer-dependent partitions in which the same writer may appear in both training and test. AugCRNN-T exceeds both

TABLE III
WORD-LEVEL ACCURACY ON IAM. WA VALUES IN PARENTHESES ARE DERIVED AS 1 − WER FROM THE CITED PAPERS.

System	Year	WA (%)	CER (%)
Aachen writer-disjoint split (same protocol as ours)			
Sueiras et al. [11]	2018	(76.20)	8.80
Kang et al. [7]	2018	(82.55)	6.88
Kang et al. [12]	2021	(84.09)	**5.79**
Kass and Vats [13]	2022	(84.60)	6.50
AugCRNN-T (proposed)	—	**84.54**	9.21
Random (writer-dependent) splits			
Dutta et al. [16]	2018	(77.14)	11.08
Rajesh et al. [17]	2022	80.08	9.89
Our baseline family (Aachen writer-disjoint)			
CRNN-S	—	70.29	17.46
CRNN-M	—	72.56	14.06
CRNN-L	—	78.06	11.16

— Dutta et al. [16] by 7.4 pp and HWRCNet [17] by 4.5 pp — while being evaluated under the stricter protocol, so those margins are conservative.

Within our own family, accuracy grows monotonically with recurrent depth (CRNN-S → CRNN-M → CRNN-L), but the single largest jump comes from the proposed augmentation rather than from capacity: going from CRNN-M to CRNN-L adds 5.5 pp at the cost of 13 M extra parameters, whereas going from CRNN-L to AugCRNN-T adds 6.5 pp at no parameter cost at all.

C. Decoder and Ensemble Experiments

Table IV lists every decoding and ensembling variant we evaluated. These experiments were carried out in a local reproduction environment whose greedy+trigram baseline is 78.29% rather than the 84.54% of the original training environment (Section VI-C explains the discrepancy), so the informative quantity is the *relative* change within the block.

Three observations follow. First, word beam search is by far the most effective alternative decoder, recovering +4.61 pp over greedy+trigram: constraining the search to a lexicon during decoding rescues errors that post-hoc correction cannot. Second, test-time augmentation is neutral (−0.06 pp): averaging five affine views of an already normalised word crop adds no information. Third, ensembling with the weaker CRNN-S and CRNN-M models is actively harmful when the outputs are averaged before decoding (−4.65 pp), because the low-capacity members dilute the confident posteriors of the strong member; the loss is only recovered once word beam search is applied on top. No configuration exceeded the 84.54% obtained by AugCRNN-T in its native environment, which is why the simple greedy+trigram pipeline remains our headline system.

TABLE IV
DECODER AND ENSEMBLE EXPERIMENTS ($N = 5{,}338$); THE BEST ALTERNATIVE DECODER IS UNDERLINED.

Configuration	WA (%)	Δ vs. local base
AugCRNN-T (native environment)	**84.54**	—
Local reproduction environment		
AugCRNN, greedy only	74.92	−3.37
AugCRNN-T (greedy + trigram)	78.29	—
+ TTA (5 views)	78.23	−0.06
+ word beam search	82.90	+4.61
+ WBS, trigram fallback	82.90	+4.61
+ TTA + WBS	82.88	+4.59
Ensembles		
CRNN-M + AugCRNN, greedy	68.12	−10.17
CRNN-M + AugCRNN-T, trigram	73.64	−4.65
CRNN-M + AugCRNN-T + WBS	82.62	+4.33
CRNN-S+M+AugCRNN (1:2:3), greedy	69.35	−8.94
CRNN-S+M+AugCRNN-T (1:2:3), trigram	74.45	−3.84

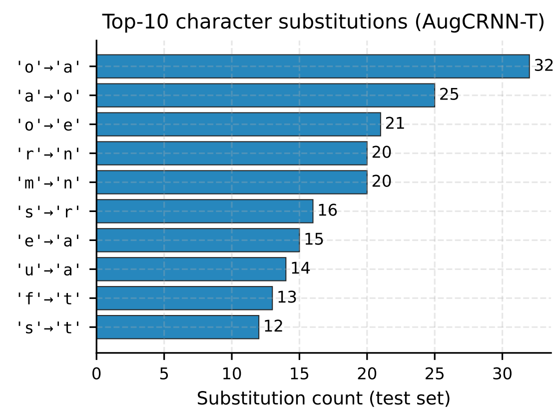

Fig. 4. Ten most frequent character substitutions of AugCRNN-T (true → predicted).

D. Error Analysis

Fig. 4 shows the ten most frequent character substitutions of AugCRNN-T. Two patterns dominate. The first is capitalisation: upper-case letters are predicted as their lower-case counterparts, which is unsurprising because case is often ambiguous in an isolated crop with no sentence context. The second is confusion between visually similar lower-case glyphs. Together these two families account for the majority of the residual character errors, which suggests that case-sensitive augmentation and a case-aware lexical prior are the most promising directions for further gains.

VI. DISCUSSION

A. Detailed Comparison with HWRCNet

The most instructive comparison is with HWRCNet [17], because it is contemporary, word-level and built from the same components: a convolutional feature extractor, a bidirectional LSTM and a CTC output layer. The two systems nevertheless differ in three respects that explain the 3.7 pp gap.

First, *capacity and depth*. HWRCNet uses a five-layer CNN with 256 hidden units in each LSTM direction, whereas

AugCRNN-T uses seven convolutional blocks and four bidirectional layers of 512 units. Our own ablation shows that depth alone accounts for part of the difference: CRNN-M, which is closest to HWRCNet in size, reaches 72.56%, and moving to the deeper CRNN-L adds 5.5 pp.

Second, *augmentation*. HWRCNet trains on unmodified crops (compressed or otherwise); no augmentation is reported. The 6.48 pp improvement of AugCRNN-T over CRNN-L in Table II indicates that this is the larger of the two factors, and it costs no parameters.

Third, *evaluation protocol*. HWRCNet uses a random 95:5 split in which the same writer may appear on both sides, whereas our test writers are unseen. Writer-dependent evaluation is known to be the easier condition, so the true gap in generalisation is likely wider than the raw 3.7 pp suggests.

Finally, the two works optimise different objectives: HWRCNet targets recognition directly in the compressed domain, trading accuracy for the elimination of a decompression step, whereas we target maximum accuracy on decompressed crops. The comparison should therefore be read as evidence that our augmentation and lexical prior are effective, not as a claim that compressed-domain recognition is inferior.

B. Why the Simpler Pipeline Wins

Word beam search improved the local baseline substantially yet never surpassed AugCRNN-T in its native environment, and ensembling hurt. Both observations follow from the same mechanism. Once the optical model is strong, the residual errors are dominated by genuinely ambiguous or out-of-vocabulary items — proper nouns, dates and abbreviations — which a lexicon cannot resolve and which averaging over weaker models only blurs further. The trigram corrector already captures the recoverable population, namely hypotheses that lie within a small edit distance of a valid word. Additional decoding machinery therefore operates on a shrinking pool of correctable errors while introducing its own failure modes, such as forcing an out-of-vocabulary proper noun onto a spurious dictionary entry.

C. Environment Drift

Reproducing our own result on a newer software stack (Python 3.12, NumPy 2.x, PyTorch 2.10) with identical code and hyper-parameters yielded 82.65% instead of 84.54%, and evaluating the trained weights on a different GPU generation lowered the greedy+trigram figure further to 78.29%. We attribute this to three interacting causes: changes to the default random generator in NumPy 2.0, which alter the realised augmentation stream; cuDNN version differences in the LSTM kernels; and mixed-precision decisions that depend on the tensor-core generation of the GPU. The magnitude of the drift — comparable to the improvement claimed by many published methods — suggests that HTR papers should report the exact software and hardware stack alongside their accuracy, a practice we follow in Section IV.

D. Threats to Validity

The external systems in Table III were evaluated on random rather than writer-disjoint partitions, so the comparison is not perfectly controlled; we have argued that this bias works against us. The decoder experiments in Table IV were measured in the drifted environment, so their absolute values are not comparable to the headline number, only their relative ordering. Finally, our augmentation study compares the full proposed scheme against its absence; isolating the individual contribution of elastic deformation and of morphological perturbation would require separate training runs that we leave to future work.

VII. Conclusion

AugCRNN-T recognises isolated handwritten words with 84.54% accuracy on the IAM Aachen writer-disjoint test set. This is 6.48 pp above an identical network trained without our augmentation, a gain that is statistically significant and obtained at no additional parameter cost. AugCRNN-T surpasses two recent word-level CRNN systems by 7.4 and 3.7 percentage points respectively, and matches an attention-based sequence-to-sequence recogniser while remaining architecturally far simpler — and it does so under a stricter writer-disjoint protocol than any of them.

The result supports a practical conclusion: for word-level HTR, an augmentation strategy that models how handwriting physically varies, combined with a lexical prior applied after decoding, is more valuable than either additional capacity or more elaborate decoding. Our negative results reinforce this, since dictionary-constrained beam search, test-time augmentation and multi-model ensembling all failed to improve the proposed pipeline.

Future work will isolate the individual contribution of each augmentation transform, extend the approach with synthetic pre-training on rendered typography, and investigate case-aware correction, which our error analysis identifies as the largest remaining source of error.

Acknowledgements

Computations were carried out on Kaggle Notebooks. We thank the maintainers of the IAM Handwriting Database and of the Aachen split.

References

[1] U.-V. Marti and H. Bunke, “The IAM-database: an English sentence database for offline handwriting recognition,” *International Journal on Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[2] R. A. University, “IAM handwriting database aachen writer-disjoint split,” Widely used community split, distributed with the RWTH Aachen speech and language processing toolkits, 2010.

[3] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006, pp. 369–376.

[4] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[5] J. Puigcerver, "Are multidimensional recurrent layers really necessary for handwritten text recognition?" in *14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, vol. 1. IEEE, 2017, pp. 67–72.

[6] T. Bluche and R. Messina, "Gated convolutional recurrent neural networks for multilingual handwriting recognition," in *14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, vol. 1. IEEE, 2017, pp. 646–651.

[7] L. Kang, J. I. Toledo, P. Riba, M. Villegas, A. Fornés, and M. Rusiñol, "Convolve, attend and spell: an attention-based sequence-to-sequence model for handwritten word recognition," in *German Conference on Pattern Recognition (GCPR)*, ser. LNCS, vol. 11269. Springer, 2018, pp. 459–472.

[8] J. Michael, R. Labahn, T. Grüning, and J. Zöllner, "Evaluating sequence-to-sequence models for handwritten text recognition," in *International Conference on Document Analysis and Recognition (ICDAR)*. IEEE, 2019, pp. 1286–1293.

[9] A. Graves and J. Schmidhuber, "Offline handwriting recognition with multidimensional recurrent neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2008, pp. 545–552.

[10] A. F. d. S. Neto, B. L. D. Bezerra, A. H. Toselli, and E. B. Lima, "HTR-Flor++: a handwritten text recognition system based on a pipeline of optical and language models," in *Proceedings of the ACM Symposium on Document Engineering (DocEng)*, 2020.

[11] J. Sueiras, V. Ruiz, A. Sanchez, and J. F. Velez, "Offline continuous handwriting recognition using sequence to sequence neural networks," *Neurocomputing*, vol. 289, pp. 119–128, 2018.

[12] L. Kang, P. Riba, M. Villegas, A. Fornés, and M. Rusiñol, "Candidate fusion: integrating language modelling into a sequence-to-sequence handwritten word recognition architecture," *Pattern Recognition*, vol. 112, p. 107790, 2021.

[13] D. Kass and E. Vats, "AttentionHTR: handwritten text recognition based on attention encoder-decoder networks," in *15th IAPR International Workshop on Document Analysis Systems (DAS)*, ser. LNCS, vol. 13237. Springer, 2022, pp. 507–522.

[14] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: transformer-based optical character recognition with pre-trained models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 13 094–13 102.

[15] G. Crosilla, L. Klic, and G. Colavizza, "Benchmarking large language models for handwritten text recognition," *Journal of Documentation*, 2025.

[16] K. Dutta, P. Krishnan, M. Mathew, and C. V. Jawahar, "Improving CNN-RNN hybrid networks for handwriting recognition," in *16th International Conference on Frontiers in Handwriting Recognition (ICFHR)*. IEEE, 2018, pp. 80–85.

[17] B. Rajesh, A. K. Gupta, A. Raj, M. Javed, and S. R. Dubey, "HWRCNet: Handwritten word recognition in JPEG compressed domain using CNN-BiLSTM network," in *International Conference on Data Analytics and Learning (DAL)*, 2022.

[18] H. Scheidl, S. Fiel, and R. Sablatnig, "Word beam search: a connectionist temporal classification decoding algorithm," in *16th International Conference on Frontiers in Handwriting Recognition (ICFHR)*. IEEE, 2018, pp. 253–258.